COIS1020H:
Programming for Computing Systems

using
Microsoft Visual C#

Chapter 1

A First Program Using C#

Why Programming

- Give me examples of systems, apps, devices, etc., that we use in everyday activities!!
  - Phones and mobile apps
  - Games
  - Banks
  - Hospitals
  - ….
- Who designed and implemented these systems??

What is Programming

- Something Mysterious?!!
  - No
- Programming is about
  - Solving problems
  - Designing instructions
  - Developing new things
- Like learning a new language
  - You talk with the computer.

In this course

- First program
- Face errors!
  - And issues
- Key success factors
  - Curiosity
  - Patience
  - Practice

GET YOUR HANDS DIRTY!!!



Programming

- Computer **program**
  - Set of instructions that tells a computer what to do
  - Also called *software*

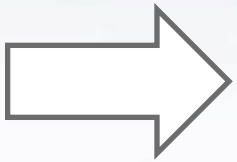
- Software comes in two broad categories
  - **System software**
    - administer the system resources.
    - Essential to run the system
  - **Application software (or *Apps*)**
    - Carry specific users' needs

Programming (cont'd.)

- **Machine language**
  - Expressed as a series of 1s and 0s
    - 1s represent switches that are on, and 0s represent switches that are off
    - Understandable to a computer
- **High-level programming languages**
  - Use reasonable terms such as “read,” “write,” or “add”
    - Instead of the sequence of on/off switches that perform these tasks
  - Has its own **syntax** (rules of the language)
    - E.g., “print” or “write”

C# Example

- int x = 5;
- int y = 3;
- int sum = x + y;



Machine Code

- 10111000 00000101
- 10111011 00000011
- 00000001 11011000

Programming (cont'd.)

- **Compiler/Interpreter/Assembler**
  - Translates programs statements into machine language (or machine code)
- **Programming logic**
  - Involves executing the various statements and procedures in the correct order
    - To produce the desired results
- **Debugging**
  - Process of removing all syntax and logical errors from the program

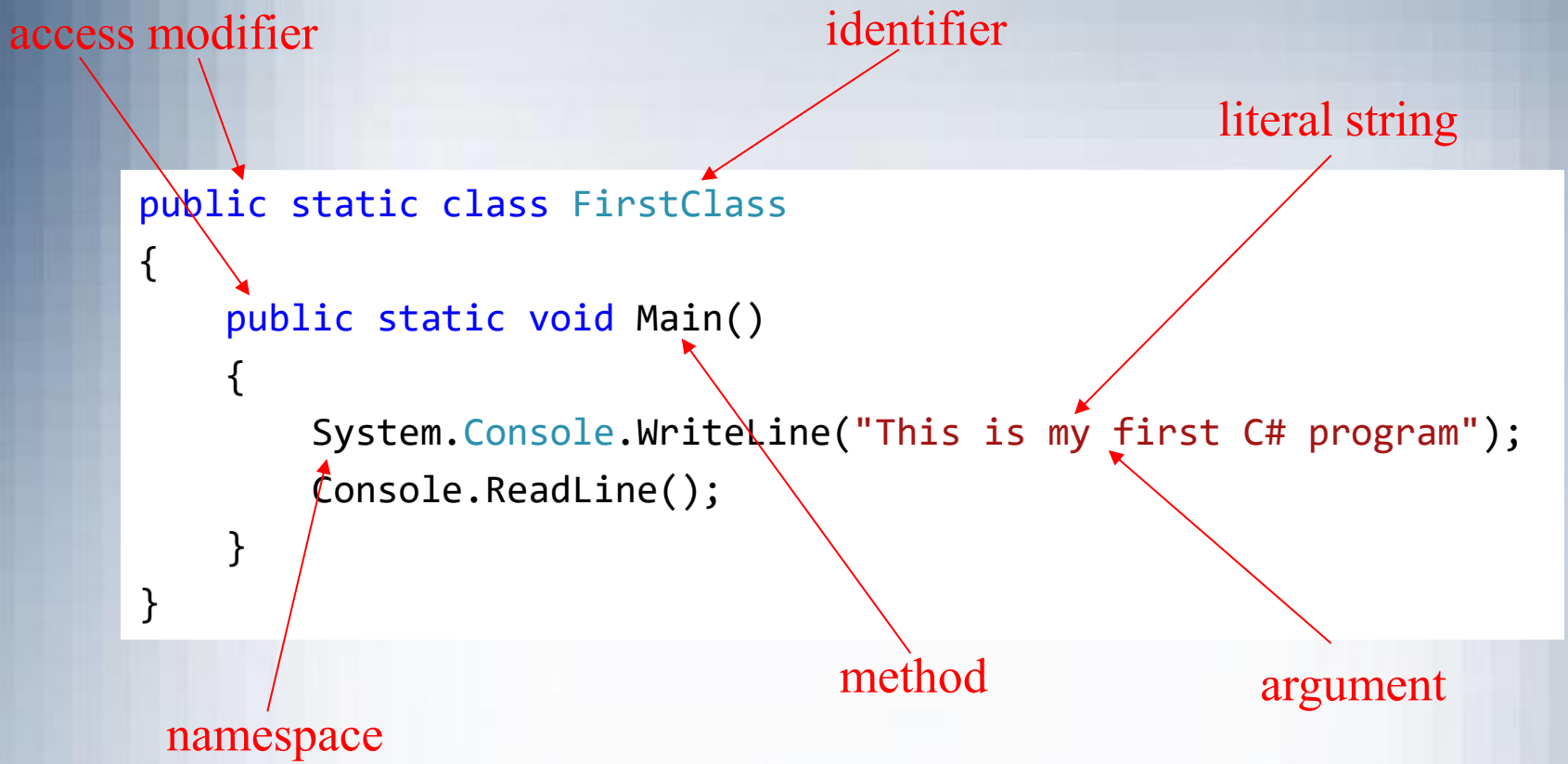
Procedural and Object-Oriented Programming

- **Procedural program**
  - Create and name computer memory locations that can hold values (**variables**)
  - A series of steps or operations to manipulate those values
- **Procedures/methods**
  - Logical units that group individual operations in a program
  - **Called** or **invoked** by other procedures/methods

The C# Programming Language

- Developed as an object-oriented language
- Part of Microsoft Visual Studio
  - Thus, contains a simple GUI interface
- Modeled after the C++ programming language
  - eliminates some of the most difficult features
- Very similar to Java syntactically

Writing a C# Program that Produces Output



Writing a C# Program that Produces Output (cont'd.)

- **Namespace**
  - Provides a way to group similar classes (more later)



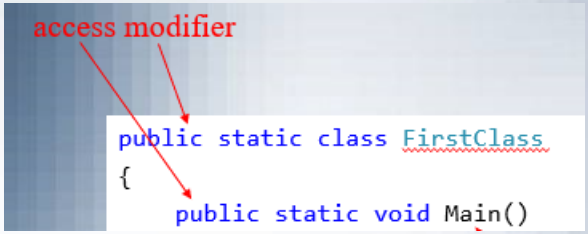
Lumber Toolbox



Mechanic Toolbox

Writing a C# Program that Produces Output (cont'd.)

- **Access modifier**
  - Defines the circumstances under which the method can be accessed



- **Keywords**
  - Predefined and reserved identifiers that have special meaning to the compiler

Writing a C# Program that Produces Output (cont'd.)

- **Whitespace**
  - Any combination of spaces, tabs, and carriage returns (blank lines)
  - Organizes your code and makes it easier to read

Writing a C# Program that Produces Output (cont'd.)

- **C# method parts**
  - **Method header**
    - Includes the method name and information about how information will be shared with the method
  - **Method body**
    - Contained within a pair of curly braces and includes all the instructions executed by the method

```
public static void Main()
   {
       System.Console.WriteLine("This is my first C# program");
   }
```

Writing a C# Program that Produces Output (cont'd.)

- The name of the method is `Main()`
  - Every console application must have a `Main()` method
- The method returns nothing as indicated by the keyword **`void`**

```
public static void Main()
    {
        System.Console.WriteLine("This is my first C# program");
    }
```

Selecting Identifiers

- **Requirements**
  - Must begin with an underscore, at sign (@), or letter
    - Letters include foreign-alphabet letters
  - Can contain only letters, digits, underscores, and the at sign
    - Not special characters such as #, $, or &
  - Cannot be a C# reserved keyword

abstract	float	return
as	for	sbyte
base	foreach	sealed
bool	goto	short
break	if	sizeof
byte	implicit	stackalloc
case	in	static
catch	int	string
char	interface	struct
checked	internal	switch
class	is	this
const	lock	throw
continue	long	true
decimal	namespace	try
default	new	typeof
delegate	null	uint
do	object	ulong
double	operator	unchecked
else	out	unsafe
enum	override	ushort
event	params	using
explicit	private	virtual
extern	protected	void
false	public	volatile
finally	readonly	while
fixed	ref	

Table 1-1 C# reserved keywords

Selecting Identifiers (cont'd.)

Class Name	Description
`Employee`	Begins with an uppercase letter
`FirstClass`	Begins with an uppercase letter, contains no spaces, and has an initial uppercase letter that indicates the start of the second word
`PushButtonControl`	Begins with an uppercase letter, contains no spaces, and has an initial uppercase letter that indicates the start of all subsequent words
`Budget2012`	Begins with an uppercase letter and contains no spaces

Table 1-2 Some valid and conventional class names in C#

The Rules:

- Starts with _, @, or letter
- contains letters, digits, _ , and the @
- Not a reserved keyword

Selecting Identifiers (cont'd.)

Class Name	Description
employee	
First_Class	
Pushbuttoncontrol	
BUDGET2013	
Public	

The Rules:

- Starts with _, @, or letter
- contains letters, digits, _ , and the @
- Not a reserved keyword

Table 1-3 Some unconventional (though legal) class names in C#

Selecting Identifiers (cont'd.)

Class Name	Description
`employee`	Unconventional as a class name because it begins with a lowercase letter
`First_Class`	Although legal, the underscore is not commonly used to indicate new words in class names
`Pushbuttoncontrol`	No uppercase characters are used to indicate the start of a new word, making the name difficult to read
`BUDGET2013`	Unconventional as a class name because it contains all uppercase letters
`Public`	Although this identifier is legal because it is different from the keyword `public`, which begins with a lowercase "p," the similarity could cause confusion

Table 1-3 Some unconventional (though legal) class names in C#

Selecting Identifiers (cont'd.)

Class Name	Description
an employee	
Push Button Control	
class	
2011Budget	
phone#	

Selecting Identifiers (cont'd.)

Class Name	Description
`an employee`	Space character is illegal
`Push Button Control`	Space characters are illegal
`class`	"class" is a reserved word
`2011Budget`	Class names cannot begin with a digit
`phone#`	The # symbol is not allowed; identifiers consist of letters, digits, underscores, or @

Table 1-4 Some illegal class names in C#

Improving Programs by Adding Program Comments

- **Program comments**
  - Non-executing statements that document a program
- **Comment out**
  - Turn a statement into a comment
- Types of comments in C#
  - Line comments //
  - Block comments /* … */

Adding Comments to a Program

- Line comment example

```
// Filename Hello.cs
// Written by <your name>
// Written on <today's date>
```

- Block comment example

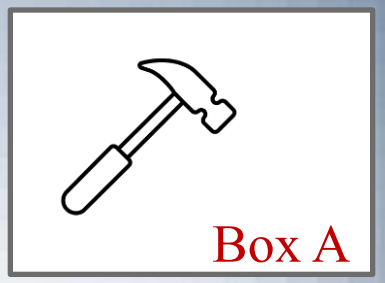
```
/* This program demonstrates the use of
the WriteLine() method to print the
message Hello, world! */
```

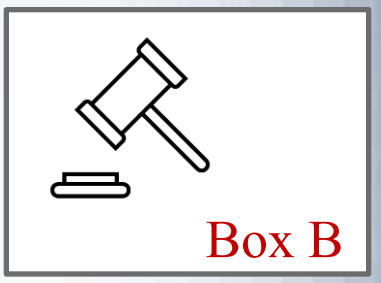
Adding Program Comments (cont'd.)

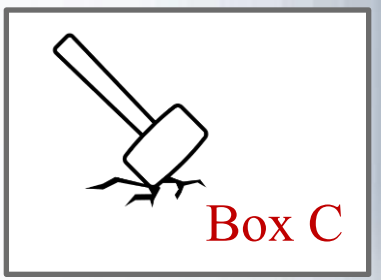
```
/* This program is written to demonstrate using comments */
public static class ClassWithOneExecutingLine
{
    public static void Main()
    {
        // The next line writes the message
        System.Console.WriteLine("Message");
        // This holds the output screen for Windows computers
        System.Console.ReadLine();
    } // End of Main
} // End of ClassWithOneExecutingLine
```

Using the `System` Namespace

- Part of the .NET framework
- Root namespace for various types





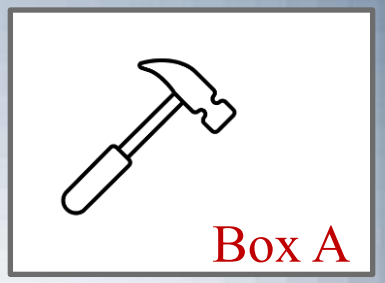


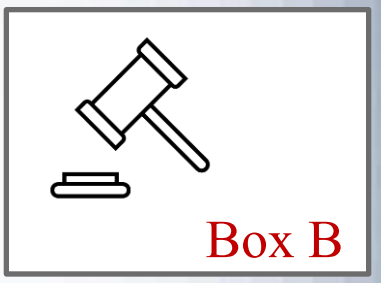

```
public static class ThreeLinesOutput
{
    public static void Main()
    {
        System.Console.WriteLine("Line one");
        System.Console.WriteLine("Line two");
        System.Console.ReadLine();
    }
}
```

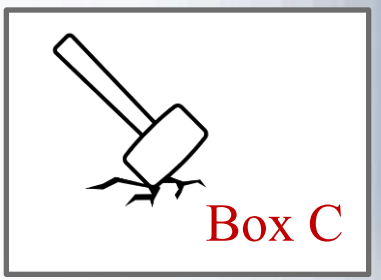
Console class is used to represent the standard input, output, and error streams for the console applications

Using the `System` Namespace (cont'd.)

```
using System; //"Using" clause or using directive
public static class ThreeLinesOutput
{
    public static void Main()
    {
        Console.WriteLine("Line one");
        Console.WriteLine("Line two");
        Console.WriteLine("Line three");
        Console.ReadLine();
    }
}
```







Writing and Compiling a C# Program

- Steps for viewing a program output
  - Compile **source code** into **intermediate language (IL)**
  - C# **just in time (JIT)** compiler translates the intermediate code into executable statements of machine code

- Can be done using either
  - A **command line environment** like Linux (not for us!)
  - An **Integrated Development Environment (IDE)** like Visual Studio

Compiling Code from within the Visual Studio IDE

- Advantages of using the Visual Studio IDE
  - Some of the code you need is already created for you
  - The code is displayed in colour
  - You can double-click an error message, and the cursor will move to the line of code that contains the error
  - Many other useful debugging tools are available
    - Will be introduced to these in the lectures and labs.

- How to install Visual Studio

Example (Part of Lab work)

- Problem: Create the following C# program (replace xxx with your name) using Microsoft Visual C#

```
using System;
public static class Lab1
{
    public static void Main()
    {
        Console.WriteLine("Hello COIS1020H, my name is xxx");
        Console.ReadLine();
    }
}
```

Summary

- A computer program is a set of instructions that tell a computer what to do
- The C# programming language is an object-oriented language
- `System.Console.WriteLine()` method
  - Produces a line of console output

Summary (cont'd.)

- You can define a C# class or variable by using any name or identifier
- Comments are non-executing statements that you add to document a program
  - Or to disable statements when you test a program
- To create a C# program, you can use the Microsoft Visual Studio environment

Syllabus